Packages and Imports

Packages in Java

A **package** is a named collection of classes and interfaces.

- A package name is a dotted name e.g. `foo.bar.baz`.
- The empty name corresponds to the default or root package.

Example I:

- The package `java.lang` contains the class `String`.
- The fully-qualified name of String is `java.lang.String`.

Example II:

- The package `java.net` contains the class `Socket`.
- The fully-qualified name of Socket is `java.net.Socket`.

Why do we want packages?

To distinguish between two concepts that share the same name.

To be able reuse of the same name for different things.

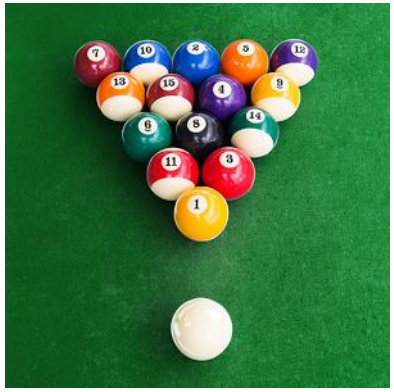
To allow different implementations of the same concept.

To organize a large collection of classes.

```
class Pool
```



```
class Pool
```



The `java.io` Package

Everything to do with input and output:

- `BufferedInputStream`
- `BufferedOutputStream`
- `BufferedReader`
- `BufferedWriter`
- …
- `File`
- …
- `FileReader`
- `FileWriter`
- …

The `java.time` Package

Everything with time

- `Clock`
- `Duration`
- `Instant`
- `LocalDate`
- `LocalDateTime`
- `LocalTime`
- `...`

Choosing Package Names

It is tradition to use **reversed domain names** as package prefixes.

- `org.junit.jupiter.api`
- `org.apache.logging.log4j`
- `ca.uwaterloo.flix`

A simple mechanism to avoid **name clashes**:

- A domain name has an owner.
- A domain name shows affiliation:
  - `dk.aau.cs.yourpackage`
- The syntax of a domain name used to be clear.